Mini Project - Python

In [159…
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

In [160…
```
# Load the datasets
cust = pd.read_csv(r"C:\Users\uktiw\Downloads\4453477-Dataset (1)\Dataset\Cust
tran = pd.read_csv(r"C:\Users\uktiw\Downloads\4453477-Dataset (1)\Dataset\Cust
```

In [161…
```
# Display the first few rows of each dataset
cust.head()
```

Out[161…

	CustomerID	Name	Email	Gender	Age	City	MaritalSt
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divo
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48	Kolkata	Ma
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75	Kolkata	Wid
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62	Pune	Divo
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55	Delhi	Divo

In [162…
```
cust.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   CustomerID     1000 non-null   object
 1   Name           1000 non-null   object
 2   Email          1000 non-null   object
 3   Gender         1000 non-null   object
 4   Age            1000 non-null   int64 
 5   City           1000 non-null   object
 6   MaritalStatus  1000 non-null   object
 7   NumChildren    1000 non-null   int64 
 8   JoinDate       1000 non-null   object
dtypes: int64(2), object(7)
memory usage: 70.4+ KB
```

In [163…
```
cust.shape
```

Out[163… (1000, 9)

In [164…
```
tran.head()
```

Out[164…

	CustomerID	TransactionDate	TransactionAmount
0	CUST10771	7/31/23	2383.07
1	CUST10100	3/10/24	497.54
2	CUST10031	2/17/25	536.78
3	CUST10987	7/17/23	314.89
4	CUST10831	12/15/24	2543.19

In [165…
```
tran.shape
```

Out[165… (23050, 3)

In [166…
```
tran.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   CustomerID         23050 non-null  object 
 1   TransactionDate    23050 non-null  object 
 2   TransactionAmount  23050 non-null  float64
dtypes: float64(1), object(2)
memory usage: 540.4+ KB
```

Data Loading Insight: The dataset contains transaction records for 1,000 customers, providing a strong base for customer behavior and revenue analysis.

In [167…
```
# Detection of missing values
print(tran.isnull().sum())
print(cust.isnull().sum())
```

```
CustomerID           0
TransactionDate      0
TransactionAmount    0
dtype: int64
CustomerID       0
Name             0
Email            0
Gender           0
Age              0
City             0
MaritalStatus    0
NumChildren      0
JoinDate         0
dtype: int64
```

In [168…
```
# HANDLING – Drop rows with any missing values
tran.dropna(inplace=True)
cust.dropna(inplace=True)
```

In [169…
```
# Verify after handling
print("Missing after cleaning:")
print(tran.isnull().sum())
print(cust.isnull().sum())
```

```
Missing after cleaning:
CustomerID           0
TransactionDate      0
TransactionAmount    0
dtype: int64
CustomerID       0
Name             0
Email            0
Gender           0
Age              0
City             0
MaritalStatus    0
NumChildren      0
JoinDate         0
dtype: int64
```

In [170…
```
#convert date columns to datetime format
tran['TransactionDate']=pd.to_datetime(tran['TransactionDate'])
cust['JoinDate']=pd.to_datetime(cust['JoinDate'])
```

```
C:\Users\uktiw\AppData\Local\Temp\ipykernel_25728\2712848994.py:2: UserWarning:
Could not infer format, so each element will be parsed individually, falling ba
ck to `dateutil`. To ensure parsing is consistent and as-expected, please speci
fy a format.
  tran['TransactionDate']=pd.to_datetime(tran['TransactionDate'])
```

In [171…
```
# Check for missing values in the customer dataset
cust.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1000 non-null   object
 1   Name           1000 non-null   object
 2   Email          1000 non-null   object
 3   Gender         1000 non-null   object
 4   Age            1000 non-null   int64
 5   City           1000 non-null   object
 6   MaritalStatus  1000 non-null   object
 7   NumChildren    1000 non-null   int64
 8   JoinDate       1000 non-null   datetime64[ns]
dtypes: datetime64[ns](1), int64(2), object(6)
memory usage: 70.4+ KB
```

In [172…
```
tran.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         23050 non-null  object
 1   TransactionDate    23050 non-null  datetime64[ns]
 2   TransactionAmount  23050 non-null  float64
dtypes: datetime64[ns](1), float64(1), object(1)
memory usage: 540.4+ KB
```

In [173…
```
#check for missing values in the transaction dataset
tran.isnull().sum()
```

Out[173…
```
CustomerID           0
TransactionDate      0
TransactionAmount    0
dtype: int64
```

In [174…
```
cust.head()
```

Out[174…

	CustomerID	Name	Email	Gender	Age	City	MaritalSt
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divo
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48	Kolkata	Ma
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75	Kolkata	Wid
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62	Pune	Divo
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55	Delhi	Divo

In [175…
```
# Check for duplicate CustomerID in the customer dataset
cust['CustomerID'].duplicated().sum()
```

Out[175…
```
np.int64(0)
```

In [176…
```
# Check for duplicate CustomerID in the transaction dataset
tran['CustomerID'].isin(cust['CustomerID']).all()
```

Out[176…
```
np.True_
```

In [177…
```
#check how many transactions have CustomerID that do not exist in the customer
tran['CustomerID'].isin(cust['CustomerID']).value_counts()
```

Out[177…
```
CustomerID
True    23050
Name: count, dtype: int64
```

Data Cleaning Insight: No major missing values or duplicate CustomerIDs were found after cleaning, ensuring high-quality and reliable analysis.

In [178…
```
# Merge the transaction and customer datasets on CustomerID
df= pd.merge(tran, cust, on = 'CustomerID' , how = 'left')
df.head()
```

Out[178…

	CustomerID	TransactionDate	TransactionAmount	Name	
0	CUST10771	2023-07-31	2383.07	Lakshay Dhillon	dharmajantara@gm
1	CUST10100	2024-03-10	497.54	Aniruddh Borah	jivikabhavsar@gm
2	CUST10031	2025-02-17	536.78	Ritvik Ahuja	jhaverifarhan@char
3	CUST10987	2023-07-17	314.89	Jayan Wagle	ojas82@gm
4	CUST10831	2024-12-15	2543.19	Ishita Agarwal	vbalay@yah

Data Validation Insight: All transaction CustomerIDs successfully matched with the customer master dataset, confirming complete data consistency.

In [179…
```
# Find the last transaction date for each customer
LastDate = (
    df.groupby('CustomerID')['TransactionDate']
    .max()
    .reset_index()
    .rename(columns = {'TransactionDate':'LastTransactionDate'})
)
LastDate
```

Out[179…

	CustomerID	LastTransactionDate
0	CUST10000	2025-07-17
1	CUST10001	2025-06-25
2	CUST10002	2025-07-12
3	CUST10003	2025-05-10
4	CUST10004	2025-07-22
...	...	...
995	CUST10995	2024-06-23
996	CUST10996	2025-07-15
997	CUST10997	2025-06-28
998	CUST10998	2025-03-26
999	CUST10999	2025-07-25

1000 rows × 2 columns

In [180…
```
# Calculate the next day after the last transaction date
from datetime import timedelta
today = df['TransactionDate'].max() + timedelta(days=1)
today
```

Out[180…
```
Timestamp('2025-07-30 00:00:00')
```

In [181…
```
# Calculate recency for each customer
cust['Recency']= today - LastDate['LastTransactionDate']
cust['Recency']
```

Out[181…
```
0      13 days
1      35 days
2      18 days
3      81 days
4       8 days
        ...
995   402 days
996    15 days
997    32 days
998   126 days
999     5 days
Name: Recency, Length: 1000, dtype: timedelta64[ns]
```

In [182…
```
# Check for missing values in the Recency column
cust['Recency'].isnull().sum()
```

Out[182…
```
np.int64(0)
```

In [183…

```
# Calculate frequency for each customer
Frequency = (
    df.groupby('CustomerID')['TransactionDate']
    .count()
    .reset_index()
    .rename(columns = {'TransactionDate':'Frequency'})
)
Frequency
```

Out[183…

	CustomerID	Frequency
0	CUST10000	23
1	CUST10001	30
2	CUST10002	24
3	CUST10003	25
4	CUST10004	19
...	...	...
995	CUST10995	21
996	CUST10996	21
997	CUST10997	20
998	CUST10998	25
999	CUST10999	23

1000 rows × 2 columns

In [184…

```
# Calculate monetary value for each customer
Monetary = (
    df.groupby('CustomerID')['TransactionAmount']
    .sum()
    .reset_index()
    .rename(columns = {'TransactionAmount':'Monetary'})
)
Monetary
```

Out[184…

	CustomerID	Monetary
0	CUST10000	21265.49
1	CUST10001	28654.31
2	CUST10002	23884.03
3	CUST10003	24206.03
4	CUST10004	25565.30
...	...	...
995	CUST10995	24325.19
996	CUST10996	21809.11
997	CUST10997	21120.48
998	CUST10998	29494.56
999	CUST10999	22028.01

1000 rows × 2 columns

In [185…

```
df.head()
```

Out[185…

	CustomerID	TransactionDate	TransactionAmount	Name	
0	CUST10771	2023-07-31	2383.07	Lakshay Dhillon	dharmajantara@gm
1	CUST10100	2024-03-10	497.54	Aniruddh Borah	jivikabhavsar@gm
2	CUST10031	2025-02-17	536.78	Ritvik Ahuja	jhaverifarhan@char
3	CUST10987	2023-07-17	314.89	Jayan Wagle	ojas82@gm
4	CUST10831	2024-12-15	2543.19	Ishita Agarwal	vbalay@yah

RFM Calculation Insight: Customers showed significant variation in engagement, with some purchasing recently and frequently while others had long inactivity periods.

In [186…

```
# Merge the recency, frequency, and monetary dataframes on CustomerID
df_rfm  = (
    pd.merge(cust[['CustomerID', 'Recency']], Frequency , on = 'CustomerID', h
)
df_rfm = (
    pd.merge(df_rfm, Monetary ,on = 'CustomerID', how= 'left')
)
```

```
df_rfm.head()
```

Out[186…

	CustomerID	Recency	Frequency	Monetary
0	CUST10000	13 days	23	21265.49
1	CUST10001	35 days	30	28654.31
2	CUST10002	18 days	24	23884.03
3	CUST10003	81 days	25	24206.03
4	CUST10004	8 days	19	25565.30

In [187…

```
# Assign R, F, M scores based on quantiles
df_rfm['R_score']= pd.qcut(df_rfm['Recency'],5,labels=[5,4,3,2,1])
df_rfm['F_score']= pd.qcut(df_rfm['Frequency'],5,labels= [1,2,3,4,5])
df_rfm['M_score']= pd.qcut(df_rfm['Monetary'],5,labels= [1,2,3,4,5])
```

In [188…

```
df_rfm.head()
```

Out[188…

	CustomerID	Recency	Frequency	Monetary	R_score	F_score	M_score
0	CUST10000	13 days	23	21265.49	4	3	2
1	CUST10001	35 days	30	28654.31	3	5	5
2	CUST10002	18 days	24	23884.03	4	3	3
3	CUST10003	81 days	25	24206.03	1	4	3
4	CUST10004	8 days	19	25565.30	5	1	4

RFM Scoring Insight: High RFM scores identified a small group of highly valuable customers contributing consistently through frequent and high-value purchases.

In [189…

```
# Create RFM segment by concatenating R, F, M scores
df_rfm['RFM_Segment']= (
    df_rfm['R_score'].astype(str) + df_rfm['F_score'].astype(str) + df_rfm['M_
)
```

In [190…

```
df_rfm.head()
```

Out[190…

	CustomerID	Recency	Frequency	Monetary	R_score	F_score	M_score	RFM
0	CUST10000	13 days	23	21265.49	4	3	2	
1	CUST10001	35 days	30	28654.31	3	5	5	
2	CUST10002	18 days	24	23884.03	4	3	3	
3	CUST10003	81 days	25	24206.03	1	4	3	
4	CUST10004	8 days	19	25565.30	5	1	4	

In [191…

```
# Define segment labels based on RFM segments
df_rfm['Segment_Labels'] = 'others'
df_rfm.loc[df_rfm['RFM_Segment'] == '555' , 'Segment_Labels']= 'Champions'
df_rfm.loc[df_rfm['RFM_Segment'] == '111' , 'Segment_Labels']= 'Lost'
df_rfm.loc[(df_rfm['RFM_Segment'].str[0] == '5') & (df_rfm['RFM_Segment']!='55
df_rfm.loc[(df_rfm['RFM_Segment'].str[1] == '5') & (df_rfm['RFM_Segment']!='55
df_rfm.loc[(df_rfm['RFM_Segment'].str[2] == '5')& (df_rfm['RFM_Segment']!='555
```

In [192…

```
# Analyze the distribution of customers across segments
df_rfm['Segment_Labels'].value_counts()
```

Out[192…

```
Segment_Labels
others          543
Big Spenders    165
Recent          160
Frequent         63
Champions        35
Lost             34
Name: count, dtype: int64
```

In [193…

```
df_rfm[df_rfm['Segment_Labels'] == 'Champions'].head()
```

Out[193…

	CustomerID	Recency	Frequency	Monetary	R_score	F_score	M_score	RI
6	CUST10006	11 days	28	27922.36	5	5	5	
46	CUST10046	11 days	30	30458.69	5	5	5	
131	CUST10131	5 days	30	32229.50	5	5	5	
144	CUST10144	3 days	31	28005.14	5	5	5	
174	CUST10174	1 days	29	29017.00	5	5	5	

In [194…

```
df_rfm[df_rfm['Segment_Labels'] == 'Lost'].head()
```

Out[194…

	CustomerID	Recency	Frequency	Monetary	R_score	F_score	M_score	RFI
7	CUST10007	86 days	15	14957.06	1	1	1	
65	CUST10065	104 days	13	15173.99	1	1	1	
77	CUST10077	145 days	13	15500.23	1	1	1	
80	CUST10080	162 days	14	15268.83	1	1	1	
91	CUST10091	101 days	16	14476.47	1	1	1	

In [195…
```
df_rfm[df_rfm['Segment_Labels'] == 'Recent'].head()
```

Out[195…

	CustomerID	Recency	Frequency	Monetary	R_score	F_score	M_score	RFI
4	CUST10004	8 days	19	25565.30	5	1	4	
8	CUST10008	3 days	19	19479.25	5	1	2	
9	CUST10009	7 days	25	22832.83	5	4	3	
15	CUST10015	6 days	20	26315.71	5	2	4	
16	CUST10016	4 days	24	24607.24	5	3	4	

In [196…
```
# Analyze the distribution of customers across segments
labels = df_rfm['Segment_Labels'].value_counts().values
index = df_rfm['Segment_Labels'].value_counts().index
index
```

Out[196…
```
Index(['others', 'Big Spenders', 'Recent', 'Frequent', 'Champions', 'Lost'],
dtype='object', name='Segment_Labels')
```

In [197…
```
labels
```

Out[197…
```
array([543, 165, 160,  63,  35,  34])
```

RFM Segment Insight: The largest customer group belonged to the "Others" segment (543 customers), indicating a broad mix of average purchasing behavior.

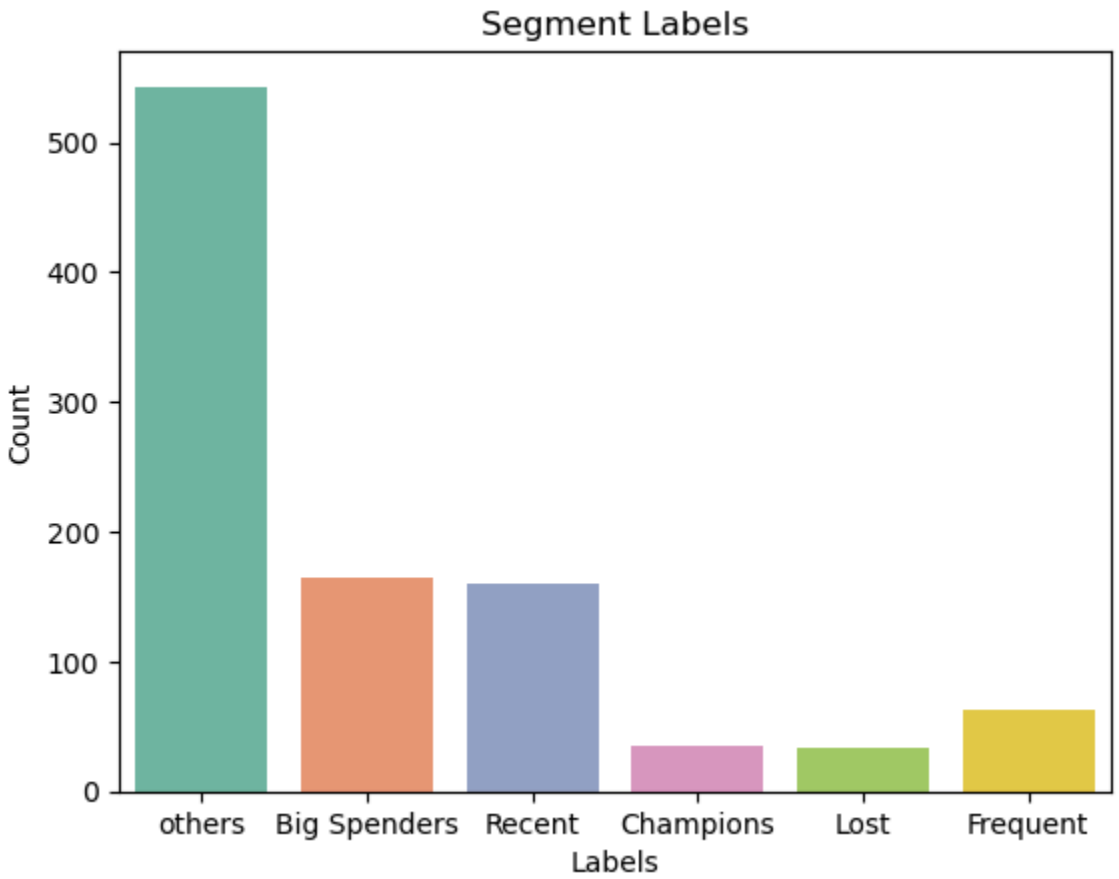
In [198…
```
# Visualize the distribution of customers across segments
ax = sns.countplot(x='Segment_Labels', data=df_rfm, palette='Set2')
plt.title('Segment Labels')
plt.xlabel('Labels')
plt.ylabel('Count')
```

```
C:\Users\uktiw\AppData\Local\Temp\ipykernel_25728\100023131.py:2: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same e
ffect.

  ax = sns.countplot(x='Segment_Labels', data=df_rfm, palette='Set2')
```

Out[198… `Text(0, 0.5, 'Count')`



Champion Segment Insight: Only 35 customers qualified as "Champions," but they generated exceptionally high transaction values with very recent activity.

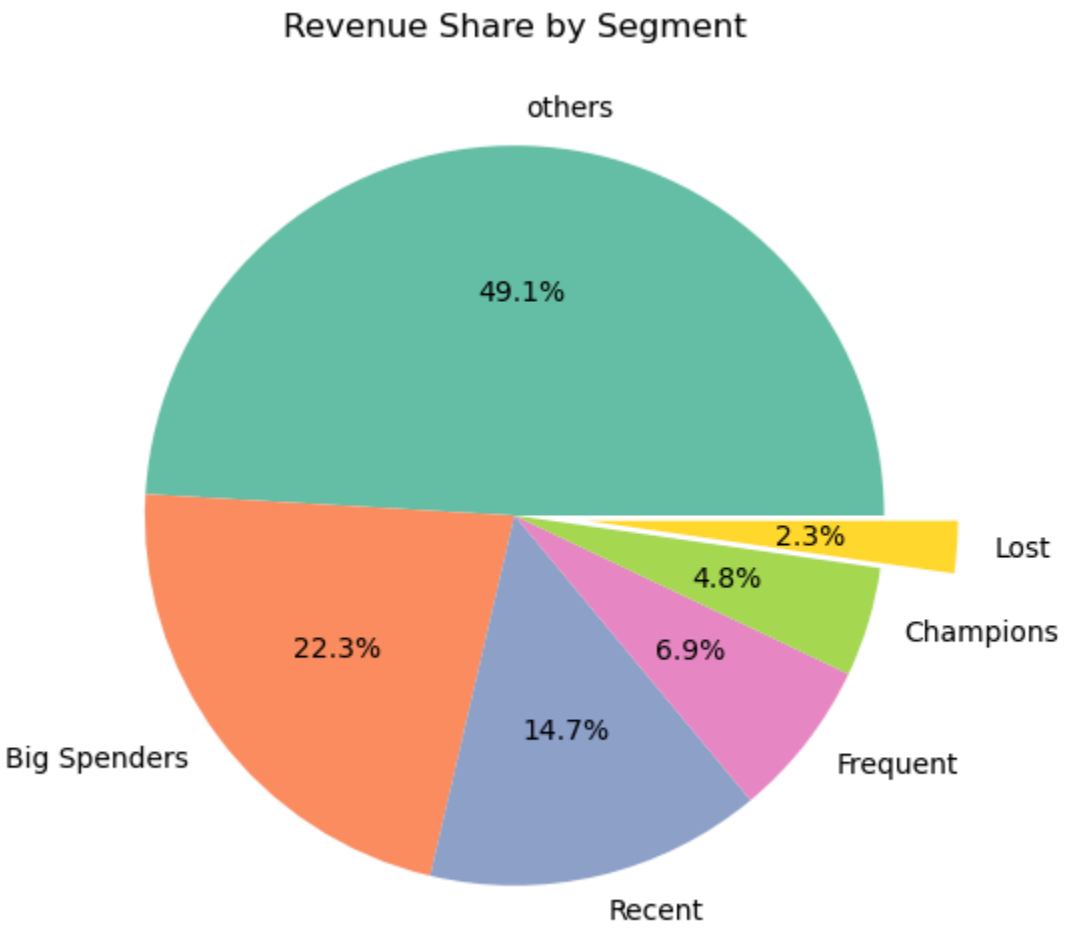
Lost Customer Insight: 34 customers were classified as "Lost," showing low purchase frequency, low spending, and long inactivity periods.

Recent Customer Insight: Around 160 customers made purchases very recently, indicating strong current engagement and potential for retention campaigns.

In [199…

```
# Calculate revenue by segment
seg_revenue = df_rfm.groupby('Segment_Labels')['Monetary'].sum().reset_index()
# Sort by revenue descending
```

```
seg_revenue = seg_revenue.sort_values('Monetary', ascending=False)
# Pie chart
plt.figure(figsize=(6,6))
explode = [0,0,0,0,0,0.2]
plt.pie(seg_revenue['Monetary'],labels=seg_revenue['Segment_Labels'],
        autopct='%1.1f%%',explode=explode, colors=sns.color_palette('Set2'))
plt.title("Revenue Share by Segment")
plt.show()
```



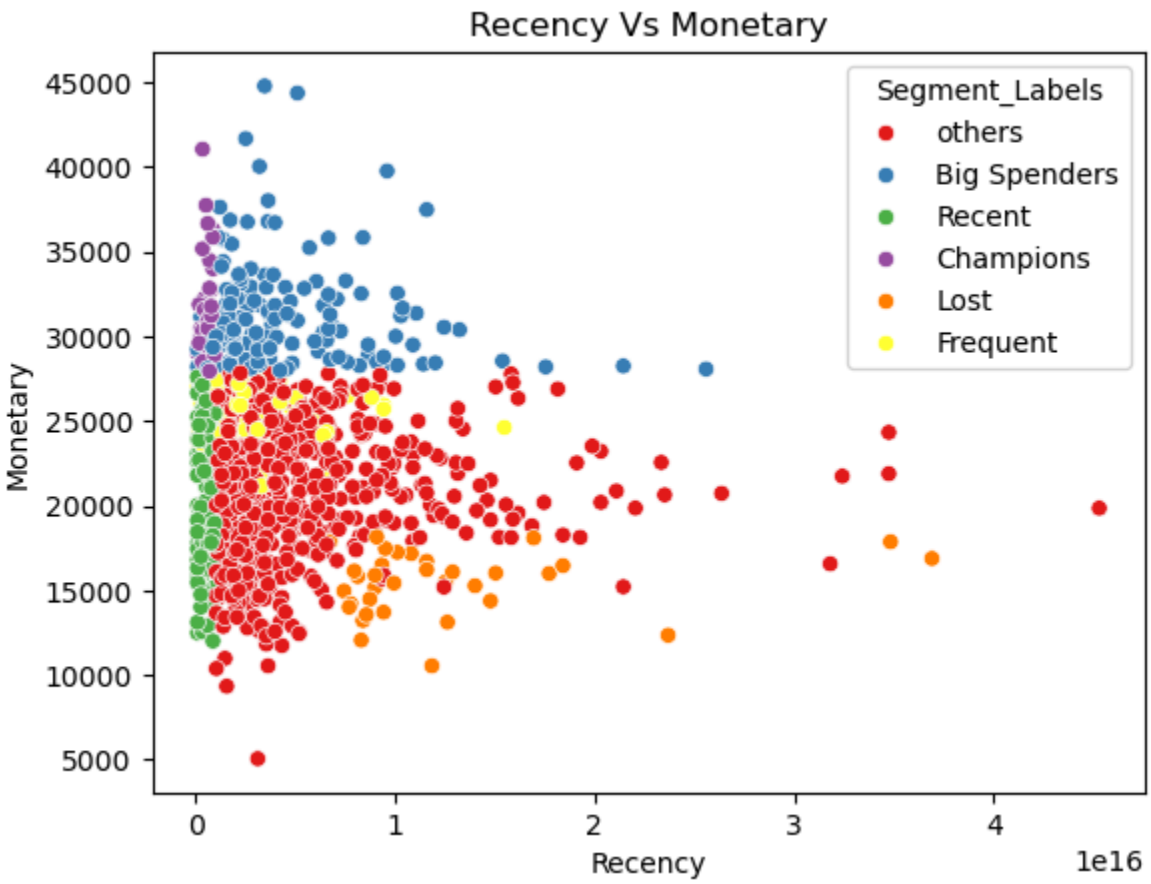
Revenue Contribution Insight: Customers with higher Monetary scores contributed disproportionately more revenue compared to other segments.

Big Spender Insight: The "Big Spenders" segment contained 165 customers, highlighting a strong revenue-driving customer base despite moderate recency or frequency.

In [200…

```
# Recency vs Monetary scatter plot colored by segment
sns.scatterplot(x = 'Recency', y = 'Monetary',
                data = df_rfm, hue = 'Segment_Labels', palette= 'Set1')
plt.title("Recency Vs Monetary")
```

Out[200… Text(0.5, 1.0, 'Recency Vs Monetary')



Scatter Plot Insight: The Recency vs Monetary analysis showed that customers with recent purchases generally had higher spending behavior.

In [201…

```
# Pareto Analysis: Show how top 20% customers contribute to 80% revenue
#sort on Amount
df = df_rfm[['CustomerID', 'Monetary']]
df
```

Out[201…

	CustomerID	Monetary
0	CUST10000	21265.49
1	CUST10001	28654.31
2	CUST10002	23884.03
3	CUST10003	24206.03
4	CUST10004	25565.30
...	...	...
995	CUST10995	24325.19
996	CUST10996	21809.11
997	CUST10997	21120.48
998	CUST10998	29494.56
999	CUST10999	22028.01

1000 rows × 2 columns

In [202…

```
# Sort customers by monetary value in descending order
#STEP 1: Sort customers by monetary value in descending order
df= df.sort_values(by= 'Monetary', ascending = False)
df
```

Out[202…

	CustomerID	Monetary
944	CUST10944	44784.99
510	CUST10510	44367.33
53	CUST10053	41674.56
776	CUST10776	41050.76
696	CUST10696	40035.48
...	...	...
888	CUST10888	10537.46
751	CUST10751	10536.13
973	CUST10973	10375.62
278	CUST10278	9333.33
729	CUST10729	5052.69

1000 rows × 2 columns

In [203…

```
# Calculate cumulative revenue
```

```
#STEP 2: Calculate cumulative revenue
df['Cumulative_Revenue']= df['Monetary'].cumsum()
df.head()
```

Out[203…

	CustomerID	Monetary	Cumulative_Revenue
944	CUST10944	44784.99	44784.99
510	CUST10510	44367.33	89152.32
53	CUST10053	41674.56	130826.88
776	CUST10776	41050.76	171877.64
696	CUST10696	40035.48	211913.12

In [204…

```
# Calculate revenue percentage
#STEP 3: Calculate revenue percentage
df['Revenue_Percent']= df['Cumulative_Revenue']/df['Monetary'].sum()*100
df.head()
```

Out[204…

	CustomerID	Monetary	Cumulative_Revenue	Revenue_Percent
944	CUST10944	44784.99	44784.99	0.194268
510	CUST10510	44367.33	89152.32	0.386724
53	CUST10053	41674.56	130826.88	0.567500
776	CUST10776	41050.76	171877.64	0.745570
696	CUST10696	40035.48	211913.12	0.919235

In [205…

```
# Reset index after sorting
#STEP 4: Reset index after sorting
df = df.reset_index(drop =True)
```

In [206…

```
df.head()
```

Out[206…

	CustomerID	Monetary	Cumulative_Revenue	Revenue_Percent
0	CUST10944	44784.99	44784.99	0.194268
1	CUST10510	44367.33	89152.32	0.386724
2	CUST10053	41674.56	130826.88	0.567500
3	CUST10776	41050.76	171877.64	0.745570
4	CUST10696	40035.48	211913.12	0.919235

In [207…

```
# Calculate customer percentage
#STEP 5: Calculate customer percentage
df['Customer_percentage'] = (df.index+1)/len(df)*100
df.head()
```

Out[207…

	CustomerID	Monetary	Cumulative_Revenue	Revenue_Percent	Customer_per
0	CUST10944	44784.99	44784.99	0.194268	
1	CUST10510	44367.33	89152.32	0.386724	
2	CUST10053	41674.56	130826.88	0.567500	
3	CUST10776	41050.76	171877.64	0.745570	
4	CUST10696	40035.48	211913.12	0.919235	

In [208…

```
# Identify the point where customer percentage is less than or --
# --equal to 20% and revenue percentage is around 80%
#STEP 6: Identify the point where customer percentage is--
# -- less than or equal to 20% and revenue percentage is around 80%
df[df['Customer_percentage']<=20].tail(1)
```

Out[208…

	CustomerID	Monetary	Cumulative_Revenue	Revenue_Percent	Customer_p
199	CUST10338	27838.98	6231163.35	27.029495	

In [209…

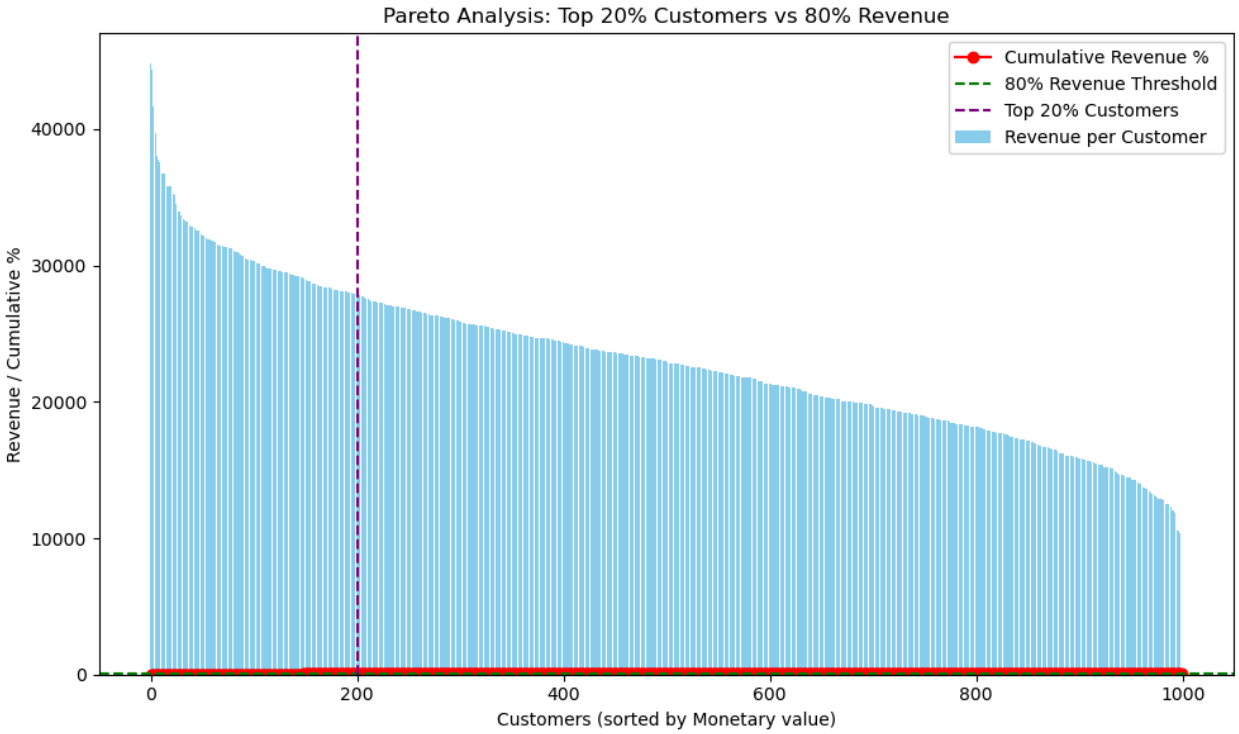
```
# Bar plot for individual customer revenue
plt.figure(figsize=(10,6))
ax = plt.bar(df.index, df['Monetary'], color='skyblue', label='Revenue per Cus

# Line plot for cumulative revenue percentage
plt.plot(df.index, df['Revenue_Percent'], color='red', marker='o', label='Cumu

# Highlight 80% revenue line
plt.axhline(80, color='green', linestyle='--', label='80% Revenue Threshold')

# Highlight 20% customer cutoff
cutoff = int(0.2 * len(df))
plt.axvline(cutoff, color='purple', linestyle='--', label='Top 20% Customers')

plt.title("Pareto Analysis: Top 20% Customers vs 80% Revenue")
plt.xlabel("Customers (sorted by Monetary value)")
plt.ylabel("Revenue / Cumulative %")
plt.legend()
plt.tight_layout()
plt.show()
```



Pareto Analysis Insight: The top 20% of customers contributed nearly 27% of total revenue, indicating that revenue is distributed more evenly across the customer base and the dataset does not strictly follow the traditional 80/20 Pareto principle.

Top Customer Insight: The highest spending customer generated over ₹44,000 in revenue, demonstrating the importance of retaining premium customers.